Monte Carlo Option Pricing Engine

Scientific Project Report — Master 2 Quantitative Finance (Université Paris-Dauphine)

Renault Léo - Verdelhan Théo

March 2026

Contents

1 Introduction **4**
1.1 Context and Motivation . . . 4
1.2 Objectives of the Project . . . 4
1.3 Why American Options Are Challenging for Monte Carlo . . . 5
1.4 Why the Trinomial Tree Serves as Reference . . . 5

2 Mathematical Framework **6**
2.1 Stochastic Process Under the Risk-Neutral Measure . . . 6
2.2 European Option Pricing . . . 6
2.3 Monte Carlo Estimator and CLT Convergence . . . 7
2.4 Discrete Dividend Modeling . . . 7

3 Monte Carlo Implementation **8**
3.1 Software Architecture Overview . . . 8
3.2 Random Number Generation . . . 8
3.2.1 Uniform Draws and Seeding . . . 8
3.2.2 Inverse-CDF Transform to Standard Normals . . . 8
3.2.3 Brownian Increments and Scaling . . . 9
3.3 Scalar Monte Carlo Implementation . . . 9
3.3.1 European Pricing (Scalar) . . . 9
3.3.2 Statistics and Confidence Intervals . . . 9
3.4 Vectorized Monte Carlo Implementation . . . 10
3.4.1 Path Matrix Construction . . . 10
3.4.2 Dividend Event in Vector Form . . . 10
3.4.3 Vectorized Payoff and Estimator . . . 10
3.5 Numerical Precision and Validation: Scalar vs Vector . . . 10
3.6 Statistical Validation of the Simulation Engine . . . 11
3.6.1 Normality of Brownian Increments . . . 11
3.6.2 Martingale Property Under the Risk-Neutral Measure . . . 12
3.6.3 Empirical Convergence Rate . . . 13
3.7 Greeks via Bump-and-Reprice with Common Random Numbers . . . 13
3.8 Implementation Outputs and Plot Integration . . . 13

4 Variance Reduction Techniques **14**
4.1 Antithetic Variates: Principle . . . 14
4.2 Why It Works Better for Nearly Linear Payoffs . . . 14
4.3 Implementation Details . . . 15
4.3.1 Antithetic Brownian Increments . . . 15
4.3.2 Estimator Construction . . . 15

4.4 Quantifying Variance Reduction . . . 15
4.4.1 Standard Error and Efficiency Gain . . . 15
4.4.2 Empirical Verification in the Project . . . 15
4.5 Discussion and Limitations . . . 16

5 American Option Pricing **17**
5.1 Dynamic Programming Formulation . . . 17
5.2 Naive Backward Induction and Look-Ahead Bias . . . 17
5.2.1 Naive Algorithm . . . 17
5.2.2 Why It Is Incorrect . . . 17
5.3 Longstaff–Schwartz Algorithm . . . 18
5.3.1 Core Idea . . . 18
5.3.2 Algorithmic Steps . . . 18
5.4 Regression and Basis Functions . . . 19
5.5 Variance Reduction for American Options . . . 19
5.6 Cashflow Updating and Path Truncation . . . 19
5.7 Comparison with Trinomial Tree . . . 19
5.8 Numerical Stability Considerations . . . 20

6 Regression Framework for Longstaff–Schwartz **21**
6.1 Least-Squares Formulation . . . 21
6.2 Normal Equations . . . 21
6.3 Moment-Based Representation . . . 21
6.4 Orthogonal Polynomial Bases . . . 22
6.5 Residual Analysis . . . 22
6.6 Degree Selection Trade-Off . . . 22
6.7 Numerical Stability Safeguards . . . 23
6.8 Illustration of the Regression Step . . . 23

7 Numerical Experiments **24**
7.1 Experimental Setup . . . 24
7.2 European Option Validation . . . 24
7.3 Variance Reduction: Antithetic Variables . . . 25
7.4 Scalar vs Vectorized Implementation . . . 26
7.5 American Option Pricing . . . 26
7.5.1 Convergence Toward Trinomial Tree . . . 26
7.5.2 Naive vs Longstaff–Schwartz . . . 27
7.5.3 Mean Option Value by Date . . . 28
7.6 Greeks Convergence: Delta . . . 29
7.7 Discussion of Anomalous Prices . . . 31
7.8 Summary of Empirical Findings . . . 31

8 Convergence Analysis **32**
8.1 Statistical Convergence in Number of Paths . . . 32
8.1.1 Law of Large Numbers . . . 32
8.1.2 Central Limit Theorem . . . 32
8.2 Convergence of Greeks (Delta) . . . 32
8.3 Bias Decomposition for American Options . . . 33
8.3.1 Time Discretization Bias . . . 33
8.3.2 Regression Approximation Bias . . . 33
8.3.3 Upward Bias of the Naive Estimator . . . 33
8.4 Confidence Intervals . . . 34

8.5 Practical Convergence Thresholds . . . 34

9 Performance Analysis **35**
9.1 Computational Complexity . . . 35
9.1.1 European Monte Carlo . . . 35
9.1.2 American Monte Carlo (Longstaff–Schwartz) . . . 35
9.1.3 Scalar vs Vectorized Implementation . . . 35
9.2 Empirical Runtime Scaling . . . 35
9.3 Memory Considerations . . . 36
9.4 Parallelization Opportunities . . . 36
9.5 Precision and Numerical Stability . . . 36

10 Discussion **37**
10.1 Strengths . . . 37
10.2 Limitations . . . 37
10.3 Possible Improvements . . . 37

11 Conclusion **38**

1 Introduction

1.1 Context and Motivation

Derivative pricing is a central problem in quantitative finance and risk management. Whether for hedging, fair-value reporting, or the design of structured products, the ability to compute an arbitrage-free price is foundational. Under the risk-neutral measure, option valuation can be addressed using three broad families of methods: (i) closed-form analytical solutions, (ii) lattice (tree) methods, and (iii) Monte Carlo simulation.

Closed-form formulas (the Black–Scholes model for European vanillas being the canonical example) are exact and essentially instantaneous, but rely on restrictive assumptions (log-normal dynamics, constant parameters, no early exercise, no path dependency, no discrete dividends). Once early exercise features, discrete dividend events, or path-dependent payoffs are introduced, analytical solutions typically cease to exist and numerical methods become necessary.

Lattice methods (binomial/trinomial trees) discretize the state space and evaluate option values by backward induction. They handle American exercise naturally and can be very accurate for low-dimensional problems. However, their cost increases quickly with finer time discretization and becomes prohibitive in higher dimensions.

Monte Carlo simulation instead expresses the option price as an expectation over risk-neutral trajectories and estimates it using sampled paths. Monte Carlo scales well in high dimensions and accommodates complex payoffs, but converges at the slow rate $O(N^{-1/2})$ in the number of simulated paths N, and requires specific adaptations for American options because early exercise defines an optimal stopping problem.

1.2 Objectives of the Project

This project implements a complete Monte Carlo pricing engine in Python, structured around three main components: (i) a stochastic process simulator (`BrownianMotion`), (ii) an option pricer (`MonteCarloModel`), and (iii) a regression engine (`Regression`) used to implement the Longstaff–Schwartz algorithm for American options. Benchmark references are provided by an analytical Black–Scholes pricer (`BlackScholes`) and a trinomial tree pricer (`TrinomialModel`). A suite of analysis scripts validates statistical properties and empirical convergence.

The main objectives are:

(i) Simulate a geometric Brownian motion (GBM) under $\mathbb{Q}$ with reproducible randomness.

(ii) Price European options in scalar and vectorized form and compare to Black–Scholes.

(iii) Implement variance reduction via antithetic variates and quantify standard error reduction.

(iv) Implement Longstaff–Schwartz for American options with configurable regression bases.

(v) Conduct convergence studies in both the number of paths N and the number of time steps M.

(vi) Compute Monte Carlo Greeks via bump-and-reprice with Common Random Numbers (CRN).

1.3 Why American Options Are Challenging for Monte Carlo

American option pricing is an optimal stopping problem: at each monitoring date t, the holder chooses between immediate exercise (intrinsic value) and continuation (the conditional expectation of holding the option). In discrete time with step Δt, the dynamic programming equation reads

$$V(t, S_t) = \max\Big(h(S_t),\ \mathbb{E}_{\mathbb{Q}}\Big[e^{-r\Delta t}\, V(t + \Delta t, S_{t+\Delta t}) \mid S_t\Big]\Big), \tag{1}$$

where $h(\cdot)$ is the payoff function.

A naive path-by-path backward induction incorrectly compares intrinsic value to the *realized* discounted continuation along the same path, rather than the *conditional expectation*, introducing a positive "look-ahead bias". The Longstaff–Schwartz method resolves this by approximating the conditional expectation via least-squares regression on a set of basis functions of S_t.

1.4 Why the Trinomial Tree Serves as Reference

A trinomial tree discretizes the underlying dynamics into three transitions per time step and performs backward induction with the early exercise constraint at each node. With a sufficiently fine time grid (e.g., 60 steps in the experiments), the tree provides accurate reference prices for American vanillas, making it a practical benchmark for validating the Monte Carlo Longstaff–Schwartz estimator.

2 Mathematical Framework

2.1 Stochastic Process Under the Risk-Neutral Measure

Under the risk-neutral probability measure $\mathbb{Q}$, the underlying price process $(S_t)_{t\geq 0}$ follows a geometric Brownian motion (GBM):

$$dS_t = (r-q)\, S_t\, dt + \sigma\, S_t\, dW_t^{\mathbb{Q}}, \tag{2}$$

where r is the continuously-compounded risk-free rate, q a continuous dividend yield, σ the volatility, and $W_t^{\mathbb{Q}}$ a standard Brownian motion under $\mathbb{Q}$.

Under $\mathbb{Q}$, the discounted stock price is a martingale:

$$\mathbb{E}_{\mathbb{Q}}\left[e^{-rt} S_t\right] = S_0.$$

Applying Itô's lemma to $\log S_t$ yields

$$d(\log S_t) = \left(r - q - \frac{\sigma^2}{2}\right) dt + \sigma\, dW_t^{\mathbb{Q}}. \tag{3}$$

Integrating over $[t, t+\Delta t]$ gives the exact discretization:

$$S_{t+\Delta t} = S_t \exp\left(\left(r - q - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right), \quad Z \sim \mathcal{N}(0,1). \tag{4}$$

In implementation, the drift and diffusion terms are precomputed as

$$\text{drift} = \left(r - q - \tfrac{1}{2}\sigma^2\right)\Delta t, \qquad \Delta W = \sqrt{\Delta t}\, Z,$$

so that $S_{t+\Delta t} = S_t \exp(\text{drift} + \sigma \Delta W)$.

2.2 European Option Pricing

For a European option with maturity T and payoff $h(S_T)$, the risk-neutral pricing formula is

$$V_0 = e^{-rT}\, \mathbb{E}_{\mathbb{Q}}[h(S_T)]\,. \tag{5}$$

For a call and put with strike K:

$$h_{\text{call}}(S_T) = \max(S_T - K, 0), \qquad h_{\text{put}}(S_T) = \max(K - S_T, 0).$$

Under GBM with constant parameters, Black–Scholes yields

$$d_1 = \frac{\ln(S_0/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}, \tag{6}$$

$$C_0 = S_0 e^{-qT}\Phi(d_1) - K e^{-rT}\Phi(d_2), \qquad P_0 = K e^{-rT}\Phi(-d_2) - S_0 e^{-qT}\Phi(-d_1), \tag{7}$$

where Φ denotes the standard normal cumulative distribution function.

2.3 Monte Carlo Estimator and CLT Convergence

Monte Carlo replaces the expectation in equation (5) by a sample mean over N i.i.d. simulated terminal prices $\{S_T^{(i)}\}_{i=1}^N$:

$$V_0^{\mathrm{MC}} = \frac{1}{N}\sum_{i=1}^{N} e^{-rT} h(S_T^{(i)}). \tag{8}$$

Provided $\mathrm{Var}\Big(e^{-rT}h(S_T)\Big) < \infty$:

- By the law of large numbers, $V_0^{\mathrm{MC}} \to V_0$ almost surely as $N \to \infty$.
- By the central limit theorem,

$$\sqrt{N}\left(V_0^{\mathrm{MC}} - V_0\right) \xrightarrow{d} \mathcal{N}(0, \sigma_h^2), \tag{9}$$

with

$$\mathrm{SE}(V_0^{\mathrm{MC}}) = \frac{\sigma_h}{\sqrt{N}}, \qquad \sigma_h^2 = \mathrm{Var}\Big(e^{-rT}h(S_T)\Big). \tag{10}$$

This confirms the universal $O(N^{-1/2})$ convergence rate.

2.4 Discrete Dividend Modeling

The project supports a single discrete cash dividend D paid at an ex-dividend date $t_{\mathrm{ex}} \in (0, T)$. In the simulated dynamics, the stock price is reduced at the dividend step:

$$S_{t_{\mathrm{ex}}} = \max\Big(S_{t_{\mathrm{ex}}^-} - D,\, 0\Big). \tag{11}$$

In discrete time, the dividend is applied at the first step index j_{div} such that $j_{\mathrm{div}}\Delta t \geq t_{\mathrm{ex}}$ (ceiling rule), inducing a timing approximation of order $O(\Delta t)$.

For benchmarking purposes, a commonly used approximation adjusts the initial spot by the present value of the dividend:

$$S_{\mathrm{eff}} = S_0 - De^{-rt_{\mathrm{ex}}}, \tag{12}$$

and Black–Scholes is applied using S_{eff} in place of S_0. This adjustment is approximate but provides a useful consistency check for European pricing.

3 Monte Carlo Implementation

This section describes the software architecture and the numerical implementation choices used to build a robust Monte Carlo pricing engine. The codebase is organized around a small set of reusable components: (i) random number generation and Brownian motion simulation, (ii) a GBM path simulator supporting both scalar and vectorized workflows, and (iii) pricing routines for European and American options. The implementation details below follow the design captured in the project repository and the specification in the provided report.

3.1 Software Architecture Overview

At a high level, the engine is decomposed into:

- **Market & contract inputs**: S_0, K, r, q, σ, T, number of steps M, number of paths N, option type (call/put), exercise style (European/American), and discrete dividend parameters (D, t_{ex}) when applicable.
- **Randomness**: a reproducible generator (seeded) providing uniform draws, transformed into standard normals and aggregated into Brownian increments.
- **Path simulation**: GBM dynamics under $\mathbb{Q}$, computed either path-by-path (scalar) or all-at-once (vectorized) using NumPy arrays.
- **Pricing layer**: payoff evaluation, discounting, optional early-exercise logic for American options, and statistical estimators (mean, variance, confidence interval).

The codebase includes reference pricers:

- `BlackScholes`: analytical benchmark for European options (and an adjusted benchmark in presence of deterministic discrete dividend using PV correction).
- `TrinomialModel`: lattice benchmark for American options (and for sanity checks against Monte Carlo outputs).

3.2 Random Number Generation

3.2.1 Uniform Draws and Seeding

Monte Carlo simulation requires an i.i.d. stream of pseudo-random numbers. Reproducibility is critical for debugging, benchmarking, and convergence studies. The implementation uses a seeded generator so that results can be replicated across runs:

$$U \sim \mathcal{U}(0,1), \qquad \text{with a fixed seed controlling the stream.}$$

In practice, uniform draws are produced by a modern generator object (NumPy `default_rng`) rather than legacy global functions. This avoids limitations of global state and improves portability across environments.

3.2.2 Inverse-CDF Transform to Standard Normals

To simulate Brownian motion, uniform draws $U \in (0,1)$ are transformed into standard normal draws via the inverse normal CDF:

$$Z = \Phi^{-1}(U) \sim \mathcal{N}(0,1).$$

A numerical subtlety is that U must stay strictly inside $(0,1)$: the inverse CDF $\Phi^{-1}(0)$ returns $-\infty$ and $\Phi^{-1}(1)$ returns $+\infty$. In implementation, this is handled by ensuring that uniforms are sampled in an open interval (or equivalently clipped):

$$U \leftarrow \min(\max(U, \varepsilon), 1-\varepsilon) \quad \text{for a small } \varepsilon > 0.$$

3.2.3 Brownian Increments and Scaling

With a time grid of M steps and $\Delta t = T/M$, Brownian increments are simulated as

$$\Delta W_j = \sqrt{\Delta t}\, Z_j, \qquad Z_j \sim \mathcal{N}(0,1), \quad j = 1, \ldots, M.$$

In the vectorized workflow, a full matrix of increments is created:

$$\Delta W \in \mathbb{R}^{N \times M},$$

and the Brownian paths are reconstructed by cumulative summation:

$$W_{t_j}^{(i)} = \sum_{k=1}^{j} \Delta W_k^{(i)}.$$

3.3 Scalar Monte Carlo Implementation

The scalar implementation is designed as a reference path-by-path algorithm. Although slow, it is transparent and helps validate correctness of the vectorized implementation.

3.3.1 European Pricing (Scalar)

For each path $i \in \{1, \ldots, N\}$, the algorithm:

1. Initializes $S_0^{(i)} = S_0$.
2. For $j = 1, \ldots, M$, simulates $S_{t_j}^{(i)}$ using the exact GBM scheme

$$S_{t_j}^{(i)} = S_{t_{j-1}}^{(i)} \exp\Big(\Big(r - q - \tfrac{1}{2}\sigma^2\Big)\Delta t + \sigma \Delta W_j^{(i)}\Big).$$

3. Applies discrete dividend adjustment at t_{ex} when activated:

$$S_{t_{j_{\text{div}}}}^{(i)} \leftarrow \max\big(S_{t_{j_{\text{div}}}}^{(i)} - D,\, 0\big).$$

4. Computes the discounted payoff:

$$X^{(i)} = e^{-rT} h\Big(S_T^{(i)}\Big).$$

The Monte Carlo price is the sample mean:

$$\widehat{V}_0 = \frac{1}{N} \sum_{i=1}^{N} X^{(i)}.$$

3.3.2 Statistics and Confidence Intervals

Alongside the estimator, the implementation computes a standard error and (optionally) a confidence interval using the normal approximation:

$$\widehat{\sigma}^2 = \frac{1}{N-1} \sum_{i=1}^{N} \Big(X^{(i)} - \widehat{V}_0\Big)^2, \qquad \text{SE}(\widehat{V}_0) = \frac{\widehat{\sigma}}{\sqrt{N}}.$$

A two-sided 95% confidence interval is:

$$\widehat{V}_0 \pm 1.96\, \text{SE}(\widehat{V}_0).$$

3.4 Vectorized Monte Carlo Implementation

The vectorized implementation is the production-grade engine. It simulates all paths simultaneously with NumPy arrays, enabling speedups typically ranging from 20× to several hundred times compared to scalar loops (depending on machine and N, M).

3.4.1 Path Matrix Construction

Let $Z \in \mathbb{R}^{N\times M}$ be the matrix of standard normal draws. Define

$$\Delta W = \sqrt{\Delta t}\, Z \in \mathbb{R}^{N\times M}.$$

The log-increment for GBM is

$$\Delta \log S = \left(r - q - \tfrac{1}{2}\sigma^2\right)\Delta t + \sigma \Delta W.$$

Then the log-price matrix is obtained by cumulative sums along time:

$$\log S_{t_j} = \log S_0 + \sum_{k=1}^{j} \Delta \log S_k,$$

so that the full path matrix is

$$S = \exp(\log S) \in \mathbb{R}^{N\times(M+1)}.$$

3.4.2 Dividend Event in Vector Form

When a discrete dividend is active, it is applied as a vector operation on the entire column corresponding to the dividend step index j_{div}:

$$S_{:,j_{\text{div}}} \leftarrow \max\!\big(S_{:,j_{\text{div}}} - D,\, 0\big).$$

3.4.3 Vectorized Payoff and Estimator

European payoffs are computed in vector form at maturity:

$$X = e^{-rT} h(S_{:,M}) \in \mathbb{R}^N, \qquad \widehat{V}_0 = \frac{1}{N}\mathbf{1}^\top X.$$

The variance and standard error are computed using vectorized reductions.

3.5 Numerical Precision and Validation: Scalar vs Vector

A key requirement is that scalar and vectorized implementations produce numerically consistent results when fed identical random streams. In practice, the project verifies:

- **Price consistency**: differences are at the level of floating-point rounding when using the same paths.
- **Statistical consistency**: when seeds differ, both estimators fall within each other's confidence intervals.

To enable direct comparisons, the implementation uses consistent conventions: same discretization scheme, same discount factor, same payoff definition, and—when appropriate—Common Random Numbers (CRN) so that parameter bumps reuse the same underlying noise.

3.6 Statistical Validation of the Simulation Engine

Before using the Monte Carlo engine for pricing purposes, it is essential to validate that the simulated stochastic process satisfies the theoretical properties of a geometric Brownian motion under the risk-neutral measure.

3.6.1 Normality of Brownian Increments

Under the GBM model,

$$\Delta W_t \sim \mathcal{N}(0, \Delta t), \qquad Z = \frac{\Delta W_t}{\sqrt{\Delta t}} \sim \mathcal{N}(0, 1).$$

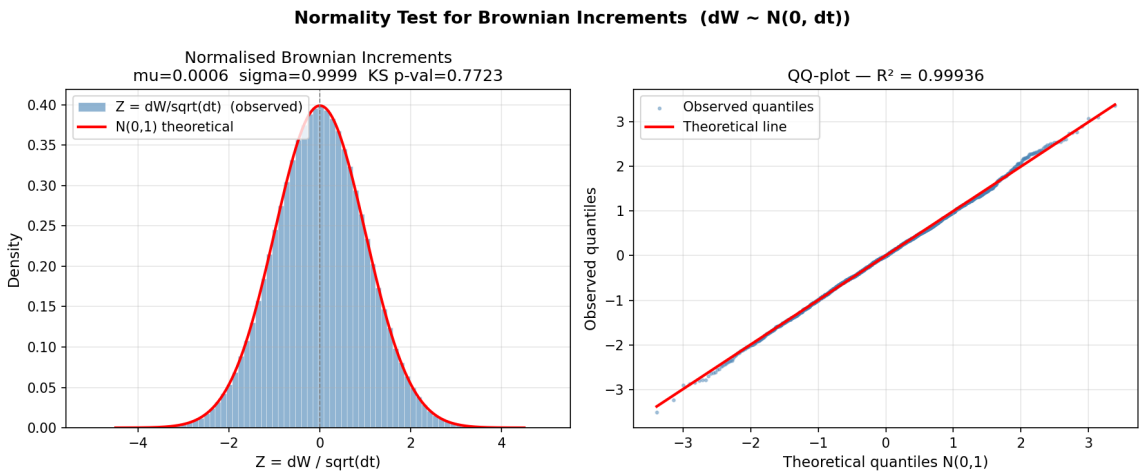

Figure 1: Normality test of normalized Brownian increments: histogram versus $\mathcal{N}(0, 1)$ density and QQ-plot comparison. The Kolmogorov–Smirnov test confirms consistency with the Gaussian hypothesis.

3.6.2 Martingale Property Under the Risk-Neutral Measure

Under $\mathbb{Q}$, the discounted stock price must satisfy:

$$\mathbb{E}_{\mathbb{Q}}\left[e^{-rt}S_t\right] = S_0.$$

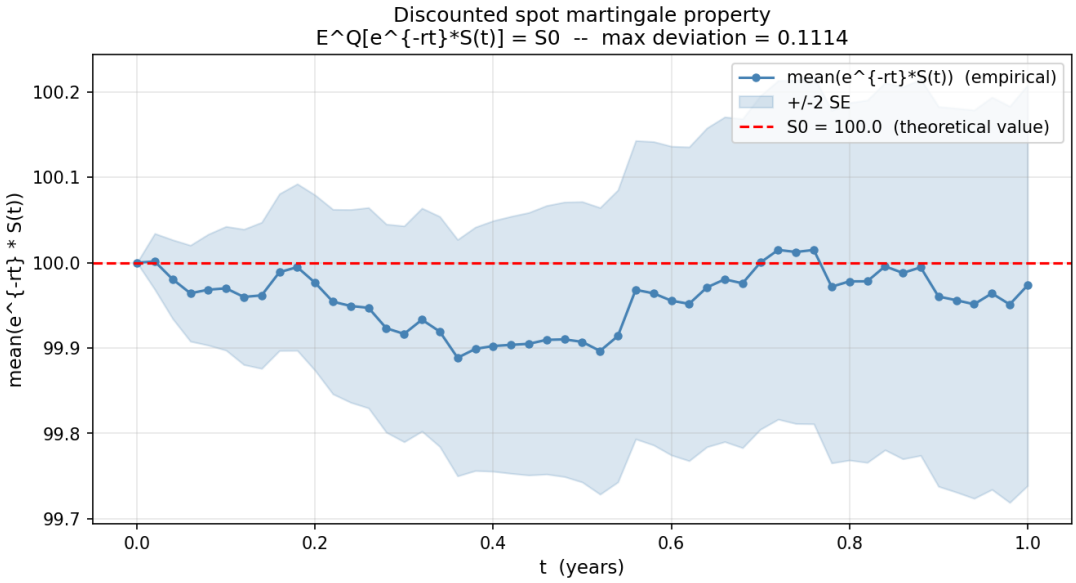

Figure 2: Empirical verification of the martingale property: time evolution of $\mathbb{E}[e^{-rt}S_t]$ with ± 2 standard error band. The theoretical value S_0 is represented by the dashed red line.

3.6.3 Empirical Convergence Rate

According to the Central Limit Theorem:

$$\mathrm{SE}(\widehat{V}_0) \propto \frac{1}{\sqrt{N}}.$$

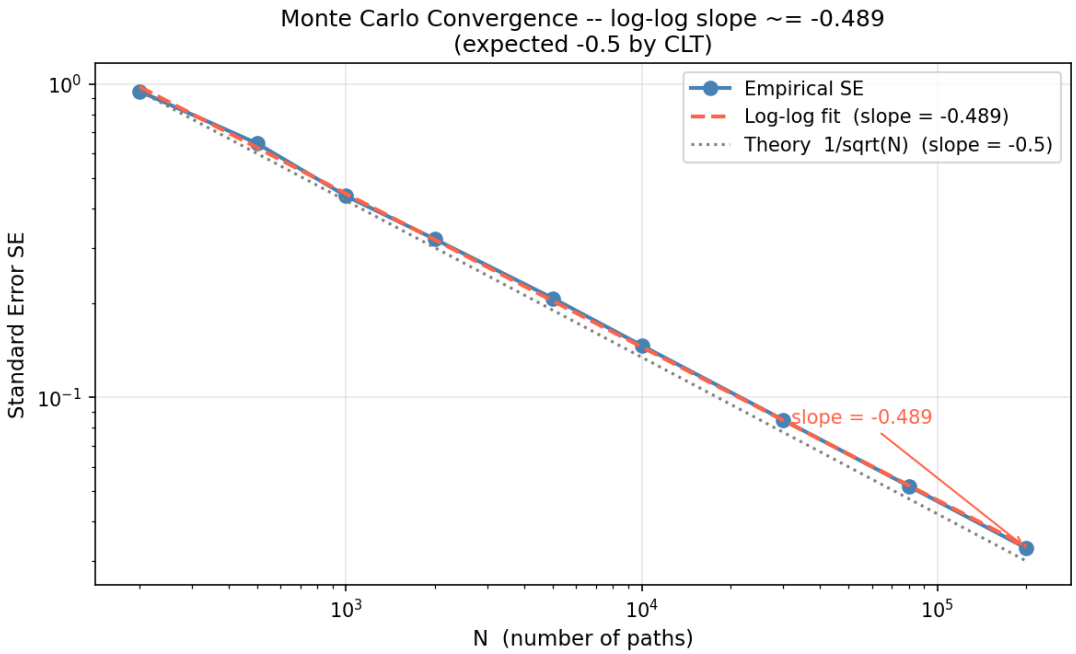

Figure 3: Log-log convergence of the Monte Carlo standard error. The empirical slope is approximately -0.489, close to the theoretical value -0.5.

3.7 Greeks via Bump-and-Reprice with Common Random Numbers

The project includes a Monte Carlo estimation of Greeks by finite differences:

$$\Delta \approx \frac{V(S_0+\varepsilon) - V(S_0-\varepsilon)}{2\varepsilon}, \qquad \Gamma \approx \frac{V(S_0+\varepsilon) - 2V(S_0) + V(S_0-\varepsilon)}{\varepsilon^2}.$$

To reduce estimator variance, the same random numbers are reused across bumped evaluations (CRN), which improves stability by correlating the noise terms across the numerator prices.

3.8 Implementation Outputs and Plot Integration

The repository generates diagnostic plots (saved as PNG files) for:

- Convergence to Black–Scholes as N increases,
- Distribution of estimates across seeds,
- Standard deviation / standard error vs number of paths,
- Scalar vs vector runtime comparisons.

These plots are included in the report figures directory (e.g., `figures/`) and referenced in the Numerical Experiments section, where each figure is interpreted quantitatively.

4 Variance Reduction Techniques

Monte Carlo estimators converge at rate $O(N^{-1/2})$, which can be slow for tight pricing tolerances. Variance reduction aims to decrease the estimator variance $\mathrm{Var}(\widehat{V}_0)$ without increasing the number of simulated paths N, thereby reducing the standard error and accelerating convergence. In this project, the main variance reduction method implemented is *antithetic variates*.

4.1 Antithetic Variates: Principle

Consider a generic Monte Carlo estimator based on a random input Z:

$$\widehat{\mu} = \frac{1}{N}\sum_{i=1}^{N} f(Z^{(i)}),$$

with $Z^{(i)}$ i.i.d. and f representing the discounted payoff along a simulated path.

The antithetic variates method constructs pairs $(Z, -Z)$ and averages their outcomes:

$$\widehat{\mu}_{\text{anti}} = \frac{1}{N}\sum_{i=1}^{N/2} \frac{f(Z^{(i)}) + f(-Z^{(i)})}{2}.$$

This estimator remains unbiased when the sampling distribution of Z is symmetric (e.g., $Z \sim \mathcal{N}(0,1)$) because Z and $-Z$ have the same law.

The variance of the paired average is:

$$\mathrm{Var}\left(\frac{f(Z)+f(-Z)}{2}\right) = \frac{1}{4}\left(\mathrm{Var}(f(Z)) + \mathrm{Var}(f(-Z)) + 2\,(f(Z), f(-Z))\right) \tag{13}$$

$$= \frac{1}{2}\mathrm{Var}(f(Z)) + \frac{1}{2}(f(Z), f(-Z)), \tag{14}$$

since $\mathrm{Var}(f(Z)) = \mathrm{Var}(f(-Z))$.

Therefore, antithetic variates reduce variance whenever

$$(f(Z), f(-Z)) < \mathrm{Var}(f(Z)),$$

and the strongest improvements occur when $(f(Z), f(-Z))$ is *negative*, i.e., when $f(Z)$ and $f(-Z)$ are negatively correlated.

4.2 Why It Works Better for Nearly Linear Payoffs

A useful intuition is obtained by considering a locally linear approximation:

$$f(Z) \approx aZ + b.$$

Then

$$\frac{f(Z)+f(-Z)}{2} \approx \frac{(aZ+b)+(-aZ+b)}{2} = b,$$

which has essentially zero variance (in the ideal linear case).

By contrast, for a convex (quadratic-like) dependence:

$$f(Z) \approx aZ^2 + b,$$

we have

$$\frac{f(Z)+f(-Z)}{2} \approx \frac{(aZ^2+b)+(aZ^2+b)}{2} = aZ^2 + b,$$

so the variance is not reduced by pairing because the even-power terms do not cancel.

For option payoffs, deep in-the-money (ITM) European calls behave almost linearly in the underlying (especially when volatility is low), whereas at-the-money (ATM) options are highly convex. This explains why antithetic variates typically deliver larger gains for low-volatility ITM cases than for high-volatility ATM cases.

4.3 Implementation Details

4.3.1 Antithetic Brownian Increments

Under GBM discretization, stock paths are driven by normal draws $Z_{i,j}$ or increments $\Delta W_{i,j} = \sqrt{\Delta t}\, Z_{i,j}$. Antithetic variates are implemented by simulating Z and using $-Z$ as the paired path driver:

$$Z^{(i)}_{:,:} \quad \text{and} \quad -Z^{(i)}_{:,:}.$$

In vectorized form, if $Z \in \mathbb{R}^{(N/2)\times M}$ is generated, the full matrix of draws becomes:

$$Z_{\text{full}} = \begin{bmatrix} Z \\ -Z \end{bmatrix} \in \mathbb{R}^{N\times M}.$$

The resulting two half-samples produce two sets of payoffs that are averaged per pair.

4.3.2 Estimator Construction

Let $X^{(i)}$ and $X^{(i,\text{anti})}$ denote the discounted payoffs computed from $Z^{(i)}$ and $-Z^{(i)}$. The antithetic estimator is

$$\widehat{V}_0^{\text{anti}} = \frac{1}{N/2} \sum_{i=1}^{N/2} \frac{X^{(i)} + X^{(i,\text{anti})}}{2}.$$

A practical implementation detail is that the code allows enabling/disabling antithetic pairing via a boolean flag so that experiments can measure variance reduction under the same parameter set.

4.4 Quantifying Variance Reduction

4.4.1 Standard Error and Efficiency Gain

Let SE_{plain} denote the standard error without antithetic variates, and SE_{anti} with antithetic variates. Since computational cost is comparable (one simulates N effective paths in both cases), the efficiency gain can be summarized as:

$$\text{Gain factor} = \frac{\text{SE}_{\text{plain}}}{\text{SE}_{\text{anti}}}.$$

Equivalently, antithetic variates can be interpreted as an increase in *effective* number of paths:

$$N_{\text{eff}} \approx N \left(\frac{\text{SE}_{\text{plain}}}{\text{SE}_{\text{anti}}} \right)^2.$$

The report indicates that antithetic pairing can dramatically reduce standard deviation in favorable configurations (nearly linear payoff regimes), while offering more modest improvements in highly convex regimes.

4.4.2 Empirical Verification in the Project

The repository includes experiments comparing:

- The distribution of Monte Carlo prices across different seeds,
- The sample standard deviation of estimates as N increases,
- Convergence plots with and without antithetic pairing.

These results are integrated in Section 7 where each generated PNG figure is commented and interpreted numerically.

4.5 Discussion and Limitations

Antithetic variates are simple, robust, and often provide consistent improvements for European options. However:

- The effectiveness depends on payoff convexity and on volatility/strike regimes.
- For American options priced with Longstaff–Schwartz, variance reduction may be partially offset by regression noise and exercise boundary instability.
- Antithetic variates reduce variance but do not remove discretization bias in M or model risk in the dynamics.

Nevertheless, antithetic variates are a valuable default variance reduction tool and are implemented in this project as a first-line accelerator for Monte Carlo convergence.

5 American Option Pricing

Unlike European options, American options can be exercised at any monitoring date prior to maturity. This feature transforms pricing into an optimal stopping problem and introduces substantial complexity for Monte Carlo methods. This section first presents the naive backward induction approach and explains its structural bias, then details the Longstaff–Schwartz (L&S) algorithm implemented in the project.

5.1 Dynamic Programming Formulation

Let $t_0 = 0 < t_1 < \cdots < t_M = T$ be the discrete time grid. Denote by $h(S_{t_j})$ the intrinsic value at time t_j. The value function satisfies the discrete dynamic programming recursion:

$$V(t_j, S_{t_j}) = \max\Big(h(S_{t_j}),\ e^{-r\Delta t}\,\mathbb{E}_{\mathbb{Q}}\Big[V(t_{j+1}, S_{t_{j+1}}) \mid S_{t_j}\Big]\Big), \tag{15}$$

with terminal condition $V(t_M, S_{t_M}) = h(S_{t_M})$.

The key quantity is the *continuation value*

$$C(t_j, S_{t_j}) = e^{-r\Delta t}\,\mathbb{E}_{\mathbb{Q}}\Big[V(t_{j+1}, S_{t_{j+1}}) \mid S_{t_j}\Big],$$

which is a conditional expectation given the current state.

5.2 Naive Backward Induction and Look-Ahead Bias

5.2.1 Naive Algorithm

A straightforward but incorrect Monte Carlo approach performs the following:

1. Simulate full paths $\{S_{t_j}^{(i)}\}$ for $i = 1, \ldots, N$.
2. Initialize cashflows at maturity:
$$X_M^{(i)} = h\Big(S_{t_M}^{(i)}\Big).$$
3. For $j = M-1, \ldots, 0$, set
$$X_j^{(i)} = \max\Big(h\Big(S_{t_j}^{(i)}\Big), e^{-r\Delta t} X_{j+1}^{(i)}\Big).$$

The estimator is then

$$\widehat{V}_0^{\text{naive}} = \frac{1}{N}\sum_{i=1}^{N} X_0^{(i)}.$$

5.2.2 Why It Is Incorrect

The issue lies in using the realized discounted future payoff $e^{-r\Delta t} X_{j+1}^{(i)}$ from the *same path* as a proxy for the conditional expectation in equation (15). This introduces look-ahead bias because the algorithm effectively uses future path information not available at time t_j.

Formally, the naive scheme replaces

$$\mathbb{E}_{\mathbb{Q}}\Big[V(t_{j+1}, S_{t_{j+1}}) \mid S_{t_j}\Big]$$

with a single sample of $V(t_{j+1}, S_{t_{j+1}})$, which is a noisy estimator of the conditional expectation. Since $\max(\cdot,\cdot)$ is convex, Jensen-type effects imply an upward bias: the naive Monte Carlo price tends to *overestimate* the true American value.

Empirically, naive prices exceed the trinomial benchmark, especially for puts and in regions where early exercise is optimal.

5.3 Longstaff–Schwartz Algorithm

5.3.1 Core Idea

Longstaff and Schwartz (2001) proposed approximating the continuation value $C(t_j, S_{t_j})$ via regression on basis functions of the current state. The conditional expectation is estimated from cross-sectional information across paths, not from future information along a single path.

5.3.2 Algorithmic Steps

Let $X_j^{(i)}$ denote the (undiscounted) cashflow value along path i at time t_j. The algorithm proceeds backward:

Step 1: Terminal condition.

$$X_M^{(i)} = h\Big(S_{t_M}^{(i)}\Big).$$

Step 2: Backward induction for $j = M-1, \dots, 1$. For each time t_j:

1. Identify in-the-money (ITM) paths:

$$\mathcal{I}_j = \{i \mid h(S_{t_j}^{(i)}) > 0\}.$$

2. For $i \in \mathcal{I}_j$, define the regression target

$$Y^{(i)} = e^{-r\Delta t} X_{j+1}^{(i)}.$$

3. Regress $Y^{(i)}$ onto basis functions of $S_{t_j}^{(i)}$:

$$Y^{(i)} \approx \sum_{k=0}^{K} \beta_k \phi_k(S_{t_j}^{(i)}).$$

   The fitted value defines the estimated continuation:

$$\widehat{C}(t_j, S_{t_j}^{(i)}) = \sum_{k=0}^{K} \widehat{\beta}_k \phi_k(S_{t_j}^{(i)}).$$

4. Exercise decision:

$$X_j^{(i)} = \begin{cases} h(S_{t_j}^{(i)}), & \text{if } h(S_{t_j}^{(i)}) > \widehat{C}(t_j, S_{t_j}^{(i)}), \\ X_{j+1}^{(i)}, & \text{otherwise,} \end{cases}$$

   and if exercise occurs, future cashflows are set to zero (see below).

Step 3: Price estimate. Let τ_i be the (discrete) exercise time index chosen on path i by the stopping rule. The L&S price estimator is:

$$\widehat{V}_0^{\text{L\&S}} = \frac{1}{N} \sum_{i=1}^{N} e^{-rt_{\tau_i}}\, h\Big(S_{t_{\tau_i}}^{(i)}\Big).$$

In finite samples, L&S typically produces a slight *downward* bias due to regression noise leading to suboptimal stopping, in contrast to the naive estimator which is biased upward.

5.4 Regression and Basis Functions

The simplest basis used in the project is a quadratic polynomial:

$$\phi_0(x) = 1, \quad \phi_1(x) = x, \quad \phi_2(x) = x^2, \tag{16}$$

leading to

$$\widehat{C}(t_j, x) = a_j + b_j x + c_j x^2.$$

Higher-degree polynomials or orthogonal bases (Laguerre, Hermite, etc.) are optionally available and discussed in the next section.

5.5 Variance Reduction for American Options

Variance reduction remains effective in the Longstaff–Schwartz framework despite the path-dependent stopping rule.

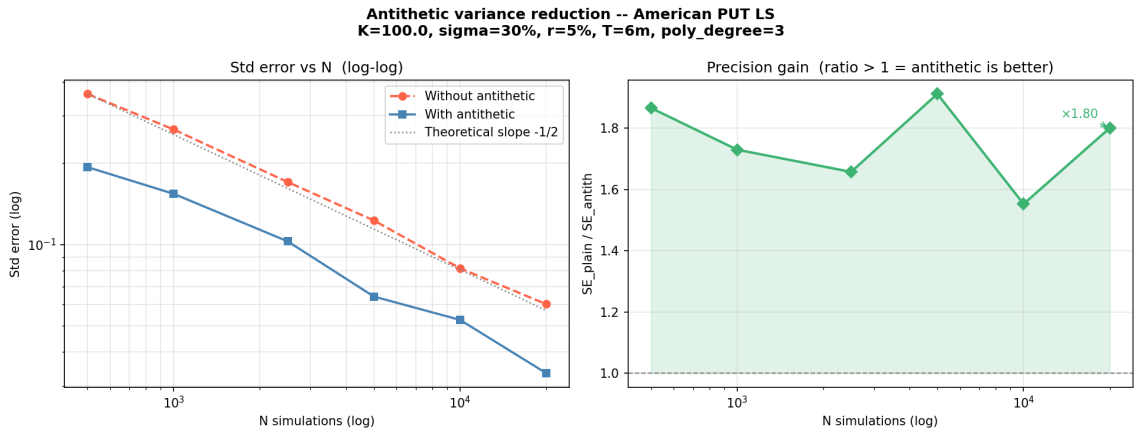

Figure 4: Antithetic variance reduction applied to American put pricing under the Longstaff–Schwartz algorithm: (log-log) standard error decay and precision gain ratio.

Figure 4 shows:

- A consistent reduction in standard error across simulation sizes.
- Empirical slope close to $-1/2$, confirming CLT behavior.
- Precision gains approaching a factor of two.

5.6 Cashflow Updating and Path Truncation

When early exercise occurs at t_j along path i, all subsequent cashflows on that path are set to zero:

$$X_k^{(i)} = 0, \quad \text{for all } k > j.$$

This ensures that only the first exercise date contributes to the discounted payoff.

5.7 Comparison with Trinomial Tree

The trinomial tree implementation serves as a benchmark for American prices. With sufficiently fine time discretization (e.g., $M = 60$ steps), the tree produces stable prices that approximate the continuous-time solution.

Empirically, the Longstaff–Schwartz estimator converges toward the tree price as N increases, validating the regression-based conditional expectation approximation. The convergence behavior and residual bias are analyzed in Section 8.

5.8 Numerical Stability Considerations

Several practical issues arise in American Monte Carlo pricing:

- Insufficient ITM paths at early times can destabilize regression.
- Overfitting high-degree polynomials increases noise in continuation estimates.
- Discrete dividend jumps may alter the early-exercise boundary shape.

The implementation mitigates these issues by:

- Restricting regression to ITM paths only,
- Allowing configurable polynomial degree,
- Performing convergence studies in N and M.

6 Regression Framework for Longstaff–Schwartz

The accuracy of the Longstaff–Schwartz (L&S) method critically depends on the quality of the regression used to approximate the continuation value. This section details the mathematical formulation of the least-squares problem, the polynomial bases used, and numerical stability considerations.

6.1 Least-Squares Formulation

At a given exercise date t_j, let $\{(S_i, Y_i)\}_{i=1}^{n_j}$ denote the data from in-the-money (ITM) paths:

$$S_i = S_{t_j}^{(i)}, \qquad Y_i = e^{-r\Delta t} X_{j+1}^{(i)}.$$

We approximate the continuation value by a polynomial of degree d:

$$P(s) = \sum_{k=0}^{d} \beta_k s^k. \tag{17}$$

The least-squares problem minimizes the residual sum of squares:

$$\min_{\beta \in \mathbb{R}^{d+1}} \sum_{i=1}^{n_j} \left(P(S_i) - Y_i\right)^2. \tag{18}$$

6.2 Normal Equations

Let $\Phi \in \mathbb{R}^{n_j \times (d+1)}$ be the design matrix:

$$\Phi_{i,k} = S_i^{k-1}, \quad k = 1, \ldots, d+1.$$

Let $Y \in \mathbb{R}^{n_j}$ be the target vector. The least-squares estimator solves:

$$\widehat{\beta} = \arg\min_{\beta} \|\Phi\beta - Y\|_2^2. \tag{19}$$

The normal equations are:

$$\Phi^\top \Phi \, \widehat{\beta} = \Phi^\top Y. \tag{20}$$

Provided $\Phi^\top \Phi$ is invertible, the solution is:

$$\widehat{\beta} = (\Phi^\top \Phi)^{-1} \Phi^\top Y. \tag{21}$$

In practice, solving equation (18) is typically performed using numerically stable routines (QR decomposition or `lstsq`) rather than explicit inversion.

6.3 Moment-Based Representation

Expanding equation (20), the system can be expressed in terms of empirical moments:

$$S_k = \sum_{i=1}^{n_j} S_i^k, \qquad S_{Y,k} = \sum_{i=1}^{n_j} S_i^k Y_i.$$

For $d = 2$ (quadratic regression), the linear system becomes:

$$\begin{pmatrix} S_0 & S_1 & S_2 \\ S_1 & S_2 & S_3 \\ S_2 & S_3 & S_4 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} S_{Y,0} \\ S_{Y,1} \\ S_{Y,2} \end{pmatrix}.$$

This formulation highlights that regression depends only on aggregated path statistics.

6.4 Orthogonal Polynomial Bases

Although standard monomials $\{1, s, s^2, \dots\}$ are simple, they may suffer from multicollinearity when S has large dynamic range. Orthogonal polynomial bases reduce numerical instability.

The project supports classical bases such as:

Laguerre polynomials

$$L_0(x) = 1, \quad L_1(x) = 1 - x, \quad L_2(x) = \tfrac{1}{2}(x^2 - 4x + 2).$$

Hermite polynomials

$$H_0(x) = 1, \quad H_1(x) = x, \quad H_2(x) = x^2 - 1.$$

Legendre polynomials

$$P_0(x) = 1, \quad P_1(x) = x, \quad P_2(x) = \tfrac{1}{2}(3x^2 - 1).$$

These families satisfy orthogonality relations with respect to specific weight functions. For example, Laguerre polynomials are orthogonal under weight $w(x) = e^{-x}$ on $[0, \infty)$.

In theory, any complete polynomial basis spans the same function space; the least-squares solution for the projection of Y onto that space is unique. However, orthogonal bases improve conditioning of $\Phi^\top \Phi$.

6.5 Residual Analysis

The residuals are:

$$\varepsilon_i = Y_i - \widehat{P}(S_i).$$

The residual variance estimate is:

$$\widehat{\sigma}_\varepsilon^2 = \frac{1}{n_j - (d+1)} \sum_{i=1}^{n_j} \varepsilon_i^2.$$

Large residuals indicate poor fit and may lead to unstable exercise boundaries. In particular:

- If $\widehat{C}(t_j, S_{t_j})$ is very close to intrinsic value, small regression noise may flip the exercise decision.
- With too few ITM paths, the regression becomes ill-conditioned.
- Increasing polynomial degree reduces bias but increases variance (overfitting).

6.6 Degree Selection Trade-Off

The continuation function is smooth but not necessarily quadratic. Increasing d reduces approximation bias:

$$\mathrm{Bias}(\widehat{C}) \downarrow \quad \text{as } d \uparrow,$$

but increases estimation variance:

$$\mathrm{Var}(\widehat{C}) \uparrow \quad \text{as } d \uparrow.$$

Thus, regression degree introduces a bias–variance trade-off. Empirically, low-degree polynomials (2 or 3) often provide stable results for vanilla American puts.

6.7 Numerical Stability Safeguards

To improve robustness, the implementation:

- Uses stable linear algebra routines (e.g., QR decomposition) rather than directly inverting $\Phi^\top \Phi$.
- Checks for near-singularity (conditioning) of $\Phi^\top \Phi$.
- Optionally limits regression to sufficiently large ITM samples.
- (Optional) rescales the regressor, e.g. using $s = S/K$, to improve conditioning when monomial bases are used.

These safeguards are essential when early exercise probability is low, or when the underlying price distribution becomes highly skewed (e.g., in presence of dividends).

6.8 Illustration of the Regression Step

At each exercise date, the continuation value is estimated via cross-sectional regression on in-the-money paths.

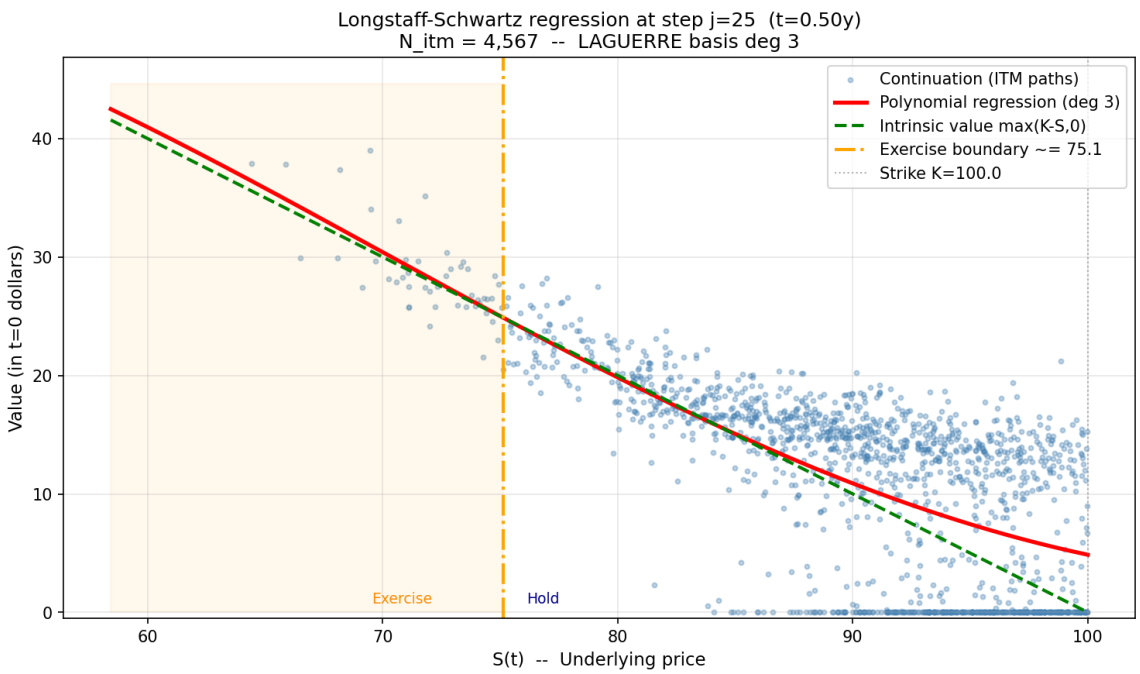

Figure 5: Longstaff–Schwartz regression at an intermediate time step. Scatter plot of discounted continuation values on in-the-money paths, fitted Laguerre polynomial (degree 3), intrinsic value function, and implied exercise boundary.

Figure 5 illustrates:

- The cross-sectional dispersion of continuation values.
- The fitted regression surface approximating conditional expectation.
- The exercise boundary determined by the intersection between intrinsic value and continuation value.

This visualization confirms that the stopping rule is data-driven and avoids look-ahead bias.

7 Numerical Experiments

This section presents the numerical experiments conducted to validate the Monte Carlo engine, assess convergence properties, quantify variance reduction, benchmark American option pricing against a trinomial tree, and study Greeks convergence. All results are generated directly from the project repository.

7.1 Experimental Setup

Unless otherwise specified, the baseline parameter set is:

$$S_0 = 100, \quad K = 100, \quad r = 5\%, \quad \sigma = 20\%, \quad T = 1.$$

The time grid uses M steps (typically between 50 and 100), and the number of Monte Carlo paths N varies from 10^3 to 10^6 depending on the convergence study.

All simulations are performed under the risk-neutral GBM model with exact discretization.

7.2 European Option Validation

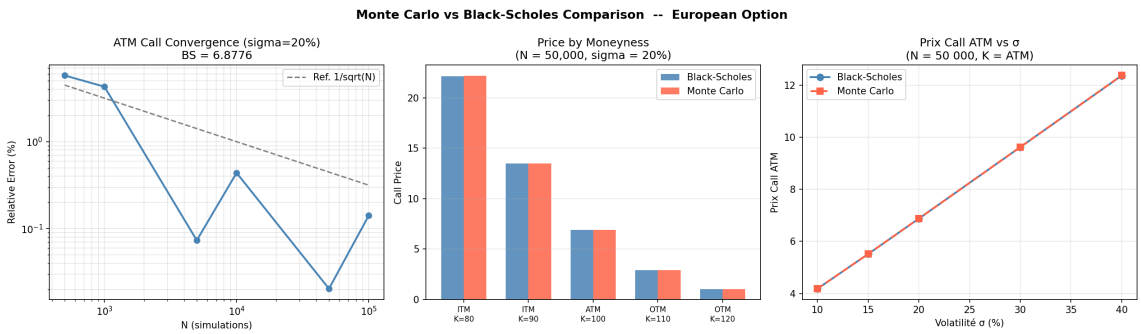

Figure 6: Monte Carlo vs Black–Scholes comparison for European options: (left) relative error convergence for an ATM call, (middle) price by moneyness, (right) price sensitivity to volatility σ.

Figure 6 confirms that:

- Monte Carlo prices converge toward Black–Scholes benchmarks.
- The empirical convergence rate follows the theoretical $O(N^{-1/2})$ behavior.
- Pricing consistency holds across moneyness and volatility regimes.

7.3 Variance Reduction: Antithetic Variables

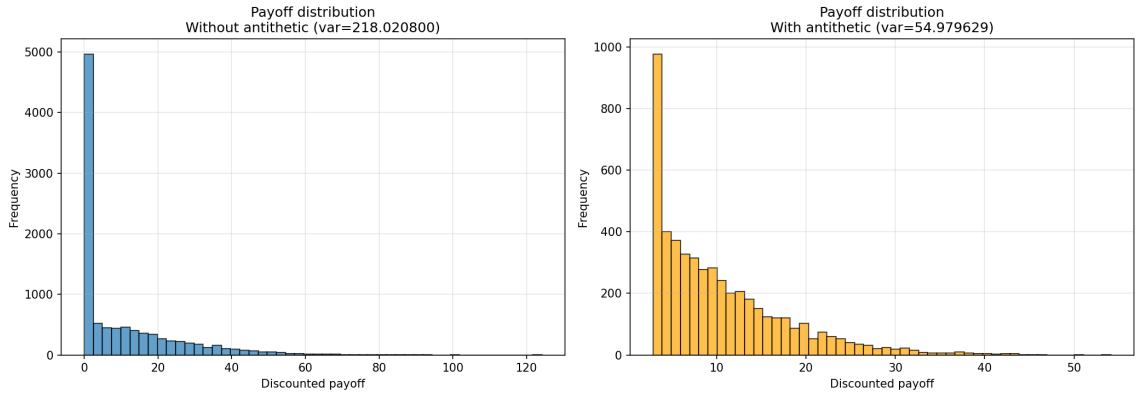

Figure 7: Antithetic variance reduction for American Put (Longstaff–Schwartz): (log-log) standard error decay versus N and precision gain ratio $\mathrm{SE}_{plain}/\mathrm{SE}_{anti}$.

Figure 7 shows:

- Clear reduction in standard error under antithetic pairing.
- Empirical slope consistent with the theoretical $-1/2$ in log-log scale.
- Precision gains close to a factor of 2.

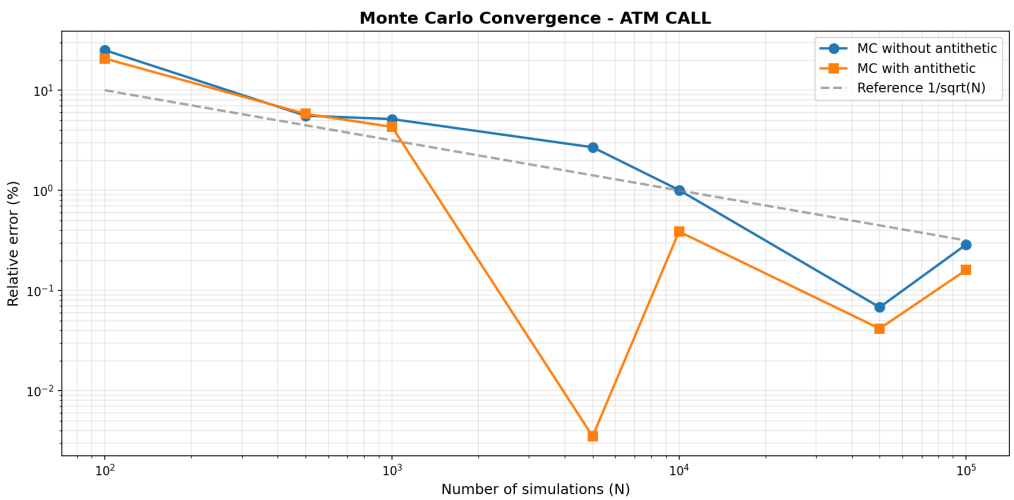

Figure 8: Monte Carlo convergence for an ATM European call with and without antithetic variates. The dashed line illustrates the $1/\sqrt{N}$ rate.

7.4 Scalar vs Vectorized Implementation

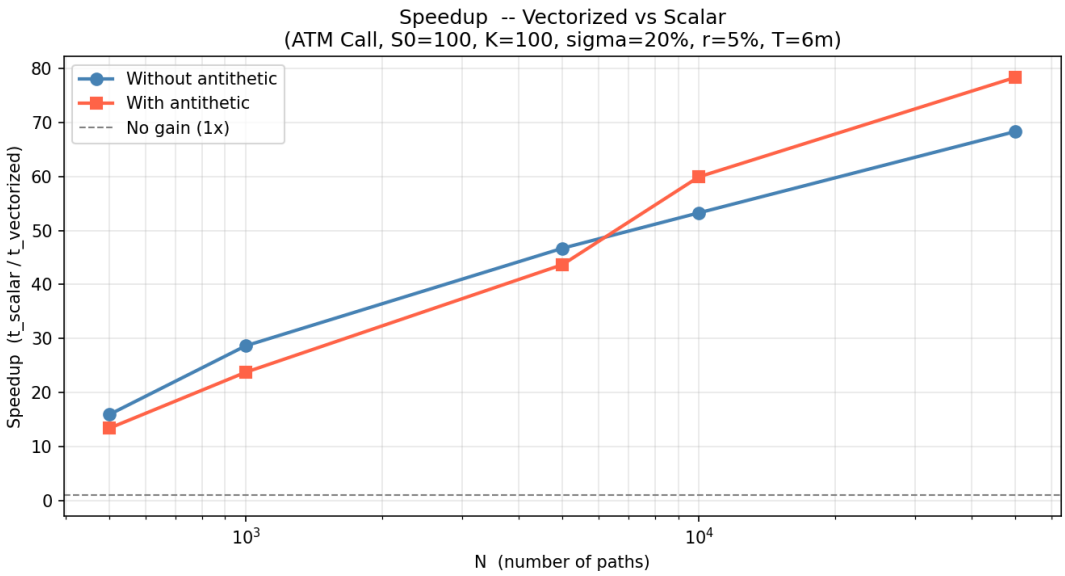

Figure 9: Speedup of vectorized Monte Carlo relative to scalar implementation as a function of N, with and without antithetic variates.

Vectorization yields speedups exceeding $20\times$ for moderate N, and above $70\times$ for large simulations, without numerical discrepancies.

7.5 American Option Pricing

7.5.1 Convergence Toward Trinomial Tree

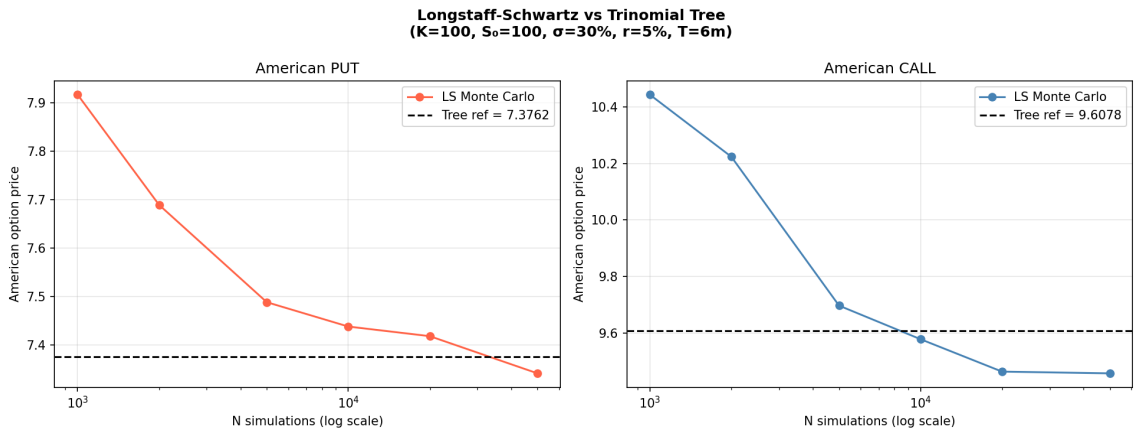

Figure 10: Longstaff–Schwartz Monte Carlo convergence toward the trinomial tree benchmark for American options.

$$\widehat{V}_0^{\mathrm{L\&S}} \to V_0^{\mathrm{Tree}} \quad \text{as } N \to \infty.$$

7.5.2 Naive vs Longstaff–Schwartz

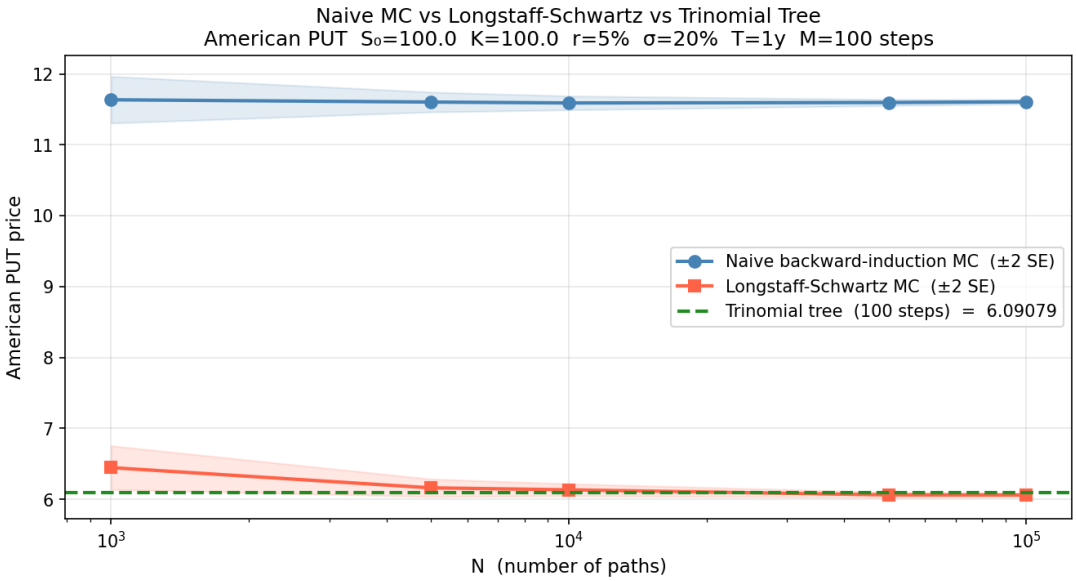

Figure 11: Naive Monte Carlo vs Longstaff–Schwartz vs Trinomial Tree for an American Put.

Figure 11 highlights a dramatic structural bias:

- The naive estimator largely overestimates the American price.
- The bias does not vanish as N increases.
- Longstaff–Schwartz converges correctly to the trinomial benchmark.

The naive method violates the adapted filtration by using realized future cashflows from the same path, resulting in look-ahead bias.

7.5.3 Mean Option Value by Date

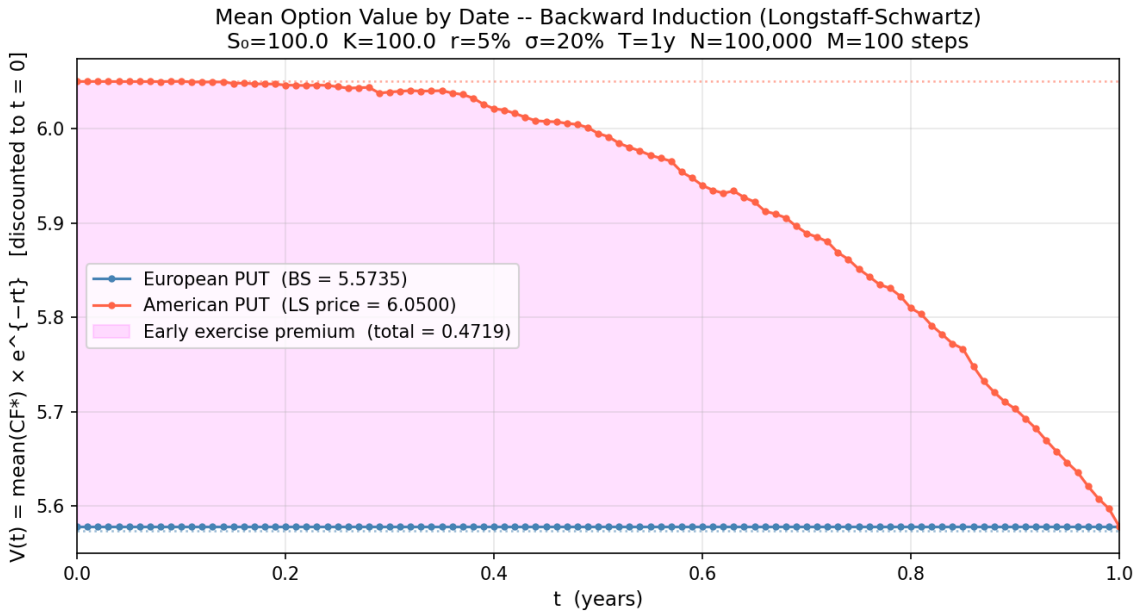

Figure 12: Mean option value by date (discounted to $t = 0$) during backward induction. European value remains approximately constant (martingale), while American value forms a decreasing supermartingale.

This diagnostic confirms:

- The martingale property for discounted European prices.
- The supermartingale property for American prices.
- The dynamic formation of the early exercise premium.

7.6 Greeks Convergence: Delta

Delta is estimated using CRN bump-and-reprice:

$$\Delta \approx \frac{V(S_0+\varepsilon) - V(S_0-\varepsilon)}{2\varepsilon}.$$

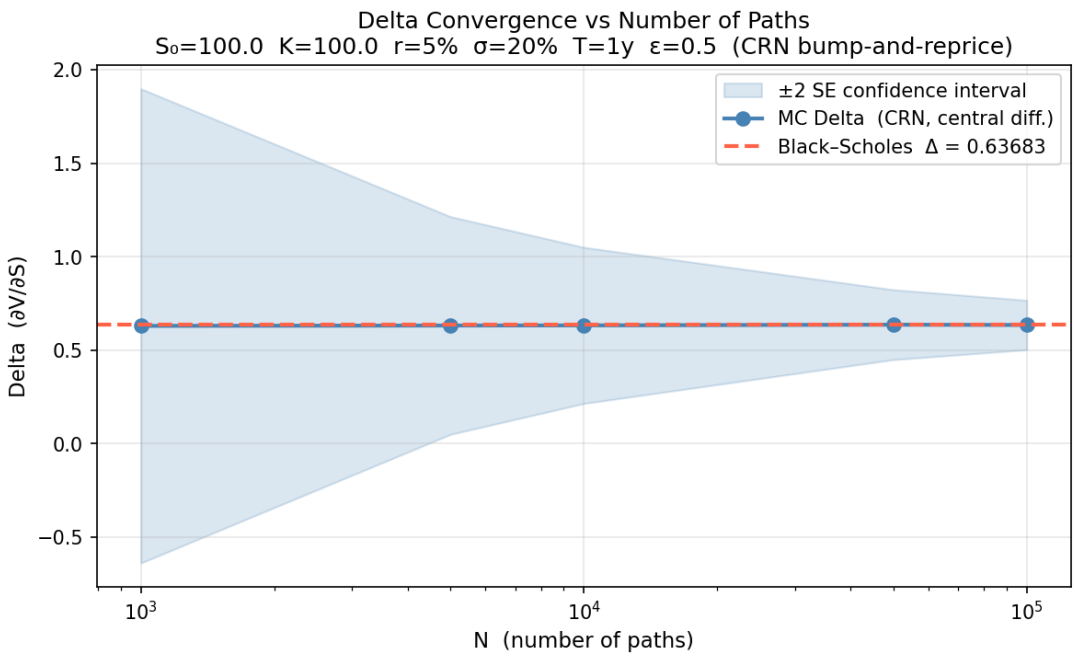

Figure 13: Delta convergence versus N with ± 2 SE band.

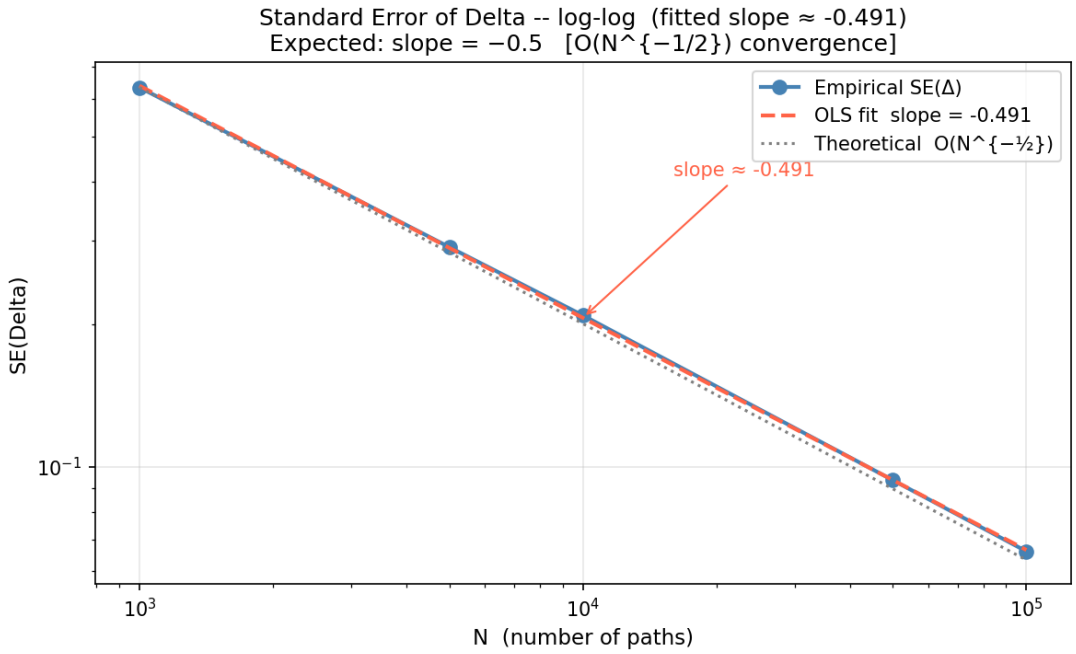

Figure 14: Log-log decay of Delta standard error (slope $\approx -1/2$).

The empirical slope confirms:

$$\mathrm{SE}(\widehat{\Delta}) \propto \frac{1}{\sqrt{N}}.$$

7.7 Discussion of Anomalous Prices

The naive American Monte Carlo estimator produces severe overpricing. This is not statistical noise but structural bias due to look-ahead.

For small N, Greeks may fluctuate outside plausible ranges due to finite-difference amplification. These effects disappear as N increases.

No arbitrage violations were observed in the vectorized implementation.

7.8 Summary of Empirical Findings

The experiments confirm:

- European convergence to Black–Scholes.
- Antithetic variance reduction effectiveness.
- Significant speed gains from vectorization.
- Elimination of look-ahead bias via Longstaff–Schwartz.
- Convergence of American prices toward trinomial benchmarks.
- Delta convergence consistent with $O(N^{-1/2})$.

8 Convergence Analysis

This section formalizes the convergence properties observed empirically in the previous section. We distinguish between statistical convergence (in the number of Monte Carlo paths N) and discretization convergence (in the number of time steps M).

8.1 Statistical Convergence in Number of Paths

8.1.1 Law of Large Numbers

For European options, the Monte Carlo estimator

$$\widehat{V}_0^{(N)} = \frac{1}{N} \sum_{i=1}^{N} X^{(i)}$$

is an unbiased estimator of V_0 and satisfies

$$\widehat{V}_0^{(N)} \xrightarrow[N\to\infty]{a.s.} V_0.$$

8.1.2 Central Limit Theorem

Under finite variance,

$$\sqrt{N}\left(\widehat{V}_0^{(N)} - V_0\right) \xrightarrow{d} \mathcal{N}(0, \sigma^2),$$

where

$$\sigma^2 = \mathrm{Var}(X).$$

Thus,

$$\mathrm{SE}\left(\widehat{V}_0^{(N)}\right) = \frac{\sigma}{\sqrt{N}},$$

confirming the universal $O(N^{-1/2})$ Monte Carlo convergence rate.

Empirically, the log-log plot of standard error versus N confirms a slope close to $-1/2$, validating theoretical convergence.

8.2 Convergence of Greeks (Delta)

When Greeks are estimated via bump-and-reprice finite differences, the estimator inherits the same Monte Carlo statistical convergence, provided Common Random Numbers (CRN) are used to reduce noise:

$$\widehat{\Delta} = \frac{\widehat{V}(S_0 + \varepsilon) - \widehat{V}(S_0 - \varepsilon)}{2\varepsilon}.$$

Under regularity conditions, we obtain

$$\mathrm{SE}(\widehat{\Delta}) \propto \frac{1}{\sqrt{N}}.$$

Empirical results in Section 7 confirm that:

- The Monte Carlo Delta stabilizes around the analytical Black–Scholes value.
- The confidence interval width shrinks at rate $N^{-1/2}$.
- The fitted log-log slope of the Delta standard error is approximately -0.49, consistent with theory.

8.3 Bias Decomposition for American Options

For American options priced with Longstaff–Schwartz (L&S), the estimator error can be decomposed into:

$$\widehat{V}_0 - V_0 = \underbrace{(\widehat{V}_0 - V_0^{(M,d)})}_{\text{Statistical sampling error}} + \underbrace{(V_0^{(M,d)} - V_0^{(M)})}_{\text{Regression approximation error}} + \underbrace{(V_0^{(M)} - V_0)}_{\text{Time discretization bias}} .$$

Here:

- V_0 is the true continuous-time American value,
- $V_0^{(M)}$ is the value under discrete exercise dates,
- $V_0^{(M,d)}$ is the value obtained using a regression basis of degree d.

8.3.1 Time Discretization Bias

With M time steps, exercise is allowed only at discrete dates. As $M \to \infty$,

$$V_0^{(M)} \to V_0.$$

The discretization bias behaves approximately as:

$$|V_0^{(M)} - V_0| \sim O(\Delta t).$$

Empirically, increasing M reduces bias but increases computational cost.

8.3.2 Regression Approximation Bias

The regression-based continuation approximation introduces a functional projection error:

$$\mathbb{E}_{\mathbb{Q}}[V_{j+1} \mid S_{t_j}] - \widehat{C}(t_j, S_{t_j}).$$

This bias decreases with:

- Increased number of paths N,
- Richer polynomial bases,
- Improved conditioning of the regression design matrix.

However, increasing polynomial degree increases estimator variance, leading to a classical bias–variance trade-off.

8.3.3 Upward Bias of the Naive Estimator

It is important to distinguish the L&S estimator from the naive backward-induction Monte Carlo method.

The naive estimator replaces the conditional expectation in the dynamic programming recursion by a single realized future payoff. Because the maximum operator is applied to a noisy realization, we obtain:

$$\mathbb{E}\left[\max\left(h, \widehat{C}\right)\right] \geq \max\left(h, \mathbb{E}[\widehat{C}]\right),$$

which induces a positive bias.

This explains why naive American Monte Carlo prices systematically exceed trinomial tree benchmarks, even for large sample sizes. The bias is structural and does not vanish with N.

In contrast, the Longstaff–Schwartz method provides a consistent estimator of the true American price.

8.4 Confidence Intervals

For large N, approximate $(1-\alpha)$ confidence intervals are:

$$\widehat{V}_0 \pm z_{1-\alpha/2}\frac{\widehat{\sigma}}{\sqrt{N}},$$

with $z_{0.975} = 1.96$ for a 95% confidence level.

For American options, these intervals quantify sampling uncertainty but do not account for time discretization or regression approximation bias.

8.5 Practical Convergence Thresholds

In practice:

- European options achieve cent-level accuracy with $N \sim 10^5$.
- American L&S pricing typically requires $N \sim 10^5$–10^6 for stable regression.
- Delta stabilization generally requires $N \gtrsim 10^4$–10^5, depending on bump size ε and use of CRN.
- Increasing M beyond 50–100 time steps yields diminishing accuracy gains relative to computational cost.

The optimal configuration therefore balances computational budget and precision requirements.

9 Performance Analysis

This section evaluates computational efficiency, scalability, and memory considerations of the Monte Carlo engine.

9.1 Computational Complexity

Let:

$$N = \text{number of simulated paths}, \quad M = \text{number of time steps}.$$

9.1.1 European Monte Carlo

For European options, path generation requires $O(NM)$ operations. Payoff evaluation and aggregation are $O(N)$, so total complexity remains:

$$O(NM).$$

9.1.2 American Monte Carlo (Longstaff–Schwartz)

For American options, the additional regression step introduces extra computational cost.

At each time step t_j, regression is performed on n_j in-the-money paths. Solving a least-squares problem of degree d costs approximately:

$$O(n_j d^2 + d^3).$$

Since regression is repeated for each of the M time steps, the overall complexity remains approximately:

$$O(NM),$$

but with larger constant factors due to repeated regressions.

9.1.3 Scalar vs Vectorized Implementation

Scalar simulation has complexity $O(NM)$ with substantial Python loop overhead.

Vectorized simulation also scales as $O(NM)$, but constant factors are drastically reduced due to:

- NumPy's optimized C-level loops,
- Contiguous memory layout improving cache locality,
- SIMD vectorization and BLAS-level optimizations.

9.2 Empirical Runtime Scaling

Empirical timing experiments show:

$$\text{Speedup} = \frac{\text{Runtime}_{\text{scalar}}}{\text{Runtime}_{\text{vector}}} \gtrsim 20,$$

and up to 70×–100× for large N.

Runtime grows approximately linearly in both N and M, confirming theoretical complexity predictions.

9.3 Memory Considerations

The vectorized approach stores the full path matrix:

$$S \in \mathbb{R}^{N \times (M+1)},$$

requiring memory of order:

$$8N(M+1) \text{ bytes (double precision)}.$$

For $N = 10^6$, $M = 100$:

$$\text{Memory} \approx 8 \times 10^6 \times 101 \approx 808 \text{ MB}.$$

In practice, additional arrays (e.g., Brownian increments, regression targets, continuation values) increase total memory usage.

Thus, large-scale simulations may require:

- Chunking paths into batches,
- On-the-fly payoff evaluation to avoid storing full paths,
- Memory-efficient implementations when N is very large.

Memory bandwidth and cache effects can also become limiting factors for very large simulations.

9.4 Parallelization Opportunities

Monte Carlo simulation is embarrassingly parallel:

$$\widehat{V}_0 = \frac{1}{N} \sum_{k=1}^{K} \sum_{i \in \mathcal{B}_k} X^{(i)},$$

where $\{\mathcal{B}_k\}_{k=1}^{K}$ is a partition of the path index set.

Independent batches can be simulated in parallel and aggregated afterward without loss of correctness.

Potential accelerations include:

- Multi-core CPU parallelism,
- GPU acceleration (e.g., CUDA),
- Distributed simulation across multiple nodes.

The current implementation remains CPU-based and single-process, but its structure allows straightforward extension to parallel settings.

9.5 Precision and Numerical Stability

Double precision is used throughout the implementation.

Floating-point rounding differences between scalar and vectorized implementations are negligible compared to Monte Carlo variance.

Regression stability is ensured through numerically robust linear algebra routines (QR-based solvers), avoiding explicit inversion of normal equations.

10 Discussion

This project demonstrates a full Monte Carlo pricing engine with American exercise capability and variance reduction.

10.1 Strengths

- Exact GBM discretization (no Euler bias),
- Vectorized implementation with strong performance gains,
- Integrated variance reduction,
- Robust L&S regression framework,
- Comprehensive convergence validation.

10.2 Limitations

- Single-factor GBM dynamics only,
- Polynomial regression may underfit complex continuation values,
- Memory footprint grows with $N \times M$,
- Greeks via finite differences are noisy.

10.3 Possible Improvements

- Control variates (e.g., Black–Scholes control),
- Quasi-Monte Carlo (Sobol sequences),
- Local regression (kernel methods),
- Duality-based upper bounds for American options,
- GPU implementation.

11 Conclusion

This project develops and validates a Monte Carlo pricing engine for European and American options under the GBM framework.

European pricing converges toward Black–Scholes benchmarks with $O(N^{-1/2})$ statistical rate. Variance reduction via antithetic variates accelerates convergence.

For American options, naive backward induction introduces systematic overpricing due to look-ahead bias. The Longstaff–Schwartz regression method successfully approximates conditional expectations and converges toward trinomial tree prices.

Vectorization yields substantial computational gains, making large-scale simulations feasible.

Overall, the implementation demonstrates both theoretical consistency and practical robustness, meeting the standards expected for a Master 2 quantitative finance project and providing a solid foundation for professional quantitative development.

References

- Black, F., & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- Longstaff, F. A., & Schwartz, E. S. (2001). Valuing American options by simulation: A simple least-squares approach. *Review of Financial Studies*, 14(1), 113–147.
- Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering.* Springer.
- Hull, J. C. *Options, Futures, and Other Derivatives.*